

Módulo 02 - Catálogo, Estoque Físico e Mídia

Objetivo

Estudar a evolução do catálogo quando ele deixa de ser apenas uma lista de produtos e passa a representar disponibilidade real, categorias, campos públicos seguros e imagens validadas.

Commits Estudados

- `bae215b` - adiciona `UnidadeBrinquedo` e contagem disponível.
- `9fdde91` - separa status do catálogo do estoque.
- `70dc172` - oculta brinquedos inativos do catálogo.
- `187e056` - adiciona categorias.
- `272da36` - oculta campos internos da API pública.
- `62bf4f2` - adiciona imagens seguras aos brinquedos.

Aula 1 - Estoque Por Unidade Física

O commit `bae215b` introduz `UnidadeBrinquedo`.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/services.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/tests.py`

Antes, a disponibilidade poderia ser confundida com um campo do brinquedo. Depois deste commit, o estoque passa a ter objetos próprios.

No estado atual, a entidade tem status operacional:

```
class UnidadeBrinquedo(models.Model):  
    class Status(models.TextChoices):
```

```
DISPONIVEL = "disponivel", "Disponivel"
RESERVADA = "reservada", "Reservada"
EM_LOCACAO = "em_locacao", "Em locacao"
HIGIENIZACAO = "higienizacao", "Higienizacao"
MANUTENCAO = "manutencao", "Manutencao"
STANDBY = "standby", "Standby"
BAIXADA = "baixada", "Baixada"
```

Decisão real de domínio:

- O negócio aluga unidades físicas, não apenas "quantidades".
- Uma cama elástica específica pode estar em manutenção enquanto outra está disponível.
- A contagem disponível deriva das unidades, não de input do frontend.

Teste para estudar:

- `test_brinquedo_com_multiplas_unidades_disponiveis_conta_corretamente`
- `test_unidades_indisponiveis_nao_contam_como_disponiveis`

Aula 2 - Catálogo Ativo Não É Estoque Disponível

O commit `9fdde91` separa o status de publicação do status físico.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/tests.py`

O campo `ativo` indica publicação no catálogo. O status da unidade indica operação física.

Essa separação evita um erro comum:

- produto inativo no site não significa unidade quebrada;
- unidade indisponível não significa produto removido do catálogo;
- admin pode tirar um produto do catálogo sem destruir o histórico operacional.

Conexão com regra do projeto:

- "Estoque é por unidade física."
- "Item devolvido vai para higienização/standby e não volta automaticamente para disponível."

Aula 3 - Correção de Catálogo Público

O commit `70dc172` é um fix importante: brinquedos inativos deixam de aparecer no catálogo público.

Código para ler:

- `backend/catalogo/services.py`
- `backend/catalogo/views.py`
- `backend/catalogo/tests.py`

No estado atual, a regra vive no service:

```
def list_public_catalog():  
    return Brinquedo.objects.select_related("categoria").filter(ativo=True)
```

O detalhe de engenharia:

- Não basta esconder no frontend.
- A API pública precisa filtrar no backend.
- O teste prova que o detalhe de brinquedo inativo retorna `404`.

Teste para estudar:

- `test_usuario_anonimo_lista_apenas_brinquedos_ativos`
- `test_usuario_anonimo_nao_visualiza_detalhe_de_brinquedo_inativo`

Aula 4 - Categorias Como Taxonomia do Produto

O commit `187e056` adiciona `Categoria` e liga `Brinquedo` a ela.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/admin.py`
- `backend/catalogo/serializers.py`
- `backend/catalogo/services.py`

Pontos reais:

- `Categoria.slug` é único.
- `Brinquedo.categoria` usa `on_delete=PROTECT`.

- O serializer público retorna um resumo da categoria.

Por que `PROTECT` importa:

- Se uma categoria estiver em uso, o sistema não deve apagá-la silenciosamente.
- Isso preserva consistência do catálogo.

Teste para estudar:

- `test_categoria_usa_slug_unico`
- `test_api_publica_retorna_categoria_do_brinquedo`
- `test_brinquedo_sem_categoria_continua_retornando_corretamente`

Aula 5 - API Pública Não Expõe Campo Interno

O commit `272da36` oculta campos internos da API pública.

Código para ler:

- `backend/catalogo/serializers.py`
- `backend/catalogo/views.py`
- `backend/catalogo/tests.py`

Decisão real:

- Admin precisa ver e alterar `ativo`.
- Cliente não precisa receber `ativo` ou `data_cadastro` no catálogo público.

No código atual, isso aparece na separação:

- `BrinquedoPublicSerializer`
- `BrinquedoAdminSerializer`

Ponto de DRF:

- Não use o mesmo serializer para todos os públicos quando os campos têm sensibilidade diferente.
- `get_serializer_class()` deixa essa fronteira explícita.

Teste para estudar:

- `test_api_publica_listagem_nao_retorna_ativo`
- `test_api_publica_detalhe_nao_retorna_data_cadastro`

Aula 6 - Upload de Imagens Com Validação

O commit `62bf4f2` cria imagens seguras para brinquedos.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/validators.py`
- `backend/catalogo/serializers.py`
- `backend/setup/settings.py`
- `backend/setup/urls.py`

Elementos reais do commit:

- `ImagemBrinquedo` guarda imagem, texto alternativo, principal, ordem e ativo.
- `caminho_imagem_brinquedo` usa `uuid4()` para evitar nomes previsíveis.
- Existe constraint para uma imagem principal por brinquedo.
- `validar_imagem_brinquedo` bloqueia extensão inválida, SVG e arquivo falso.

Exemplo de modelagem:

```
models.UniqueConstraint(
    fields=["brinquedo"],
    condition=Q(principal=True),
    name="catalogo_uma_imagem_principal_por_brinquedo",
)
```

Ponto de segurança:

- Upload não pode confiar só na extensão.
- O backend valida tipo real da imagem.
- A API pública retorna URL, não caminho interno do arquivo.

Testes para estudar:

- `test_api_publica_nao_expoe_caminho_interno_do_arquivo`
- `test_upload_bloqueia_svg`
- `test_upload_bloqueia_extensao_falsa_que_nao_e_imagem_real`

Revisão do Módulo

Você deve conseguir responder:

- Por que `ativo` não resolve estoque?
- Por que unidade física precisa de status próprio?
- Onde a API separa serializer público de serializer admin?
- Qual risco o commit `62bf4f2` reduz?
- Que testes provam que o backend protege o catálogo público?

Exercício Prático

Abra `git show bae215b` e encontre onde `quantidade_disponivel` passa a ser calculada.

Depois abra `git show 272da36` e identifique quais campos saem da resposta pública.